МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №5

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему
«Розробка багатовіконного інтерфейсу користувача для графічного редактора об’єктів»

Виконав:
студент групи ІМ-55
Стеблевець Олександр Олександрович
Номер в списку – 26

Перевірив:
Рекечинський Д.О.

Київ 2026

Мета роботи

Отримати навички програмування багатовіконного інтерфейсу в об’єктно-орієнтованому стилі.

Завдання та варіант

Номер у журналі – 26, тому контролер редактора потрібно реалізувати як класичний Singleton. Графічний редактор з попередньої роботи необхідно доповнити окремим немодальним вікном таблиці об’єктів та збереженням фігур у файл. Також потрібно передбачити виділення й видалення фігур через таблицю та завантаження збережених фігур із файлу.

Лістинги програми

```dart
import 'src/app.dart';

Future<void> main() => launchPaint();
```

Лістинг 1 – main.dart

```dart
import 'package:desktop_multi_window/desktop_multi_window.dart';
import 'package:flutter/material.dart';
import 'package:window_manager/window_manager.dart';

import 'editor/editor_page.dart';
import 'table/table_window.dart';

Future<void> launchPaint() async {
  WidgetsFlutterBinding.ensureInitialized();
  await windowManager.ensureInitialized();
  final window = await WindowController.fromCurrentEngine();
  if (window.arguments.isEmpty) {
    runApp(_EditorApp(window: window));
    return;
  }

  runApp(MaterialApp(home: TableWindow(window: window)));
}

class _EditorApp extends StatelessWidget {
  const _EditorApp({required this.window});
  final WindowController window;

  @override
  Widget build(BuildContext context) => MaterialApp(
    title: 'Paint',
    theme: ThemeData(colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple), useMaterial3:
true),
    home: EditorPage(title: 'Paint', window: window),
  );
}
```

Лістинг 2 – app.dart

```dart
import 'dart:async';
import 'dart:io';

import 'package:desktop_multi_window/desktop_multi_window.dart';
import 'package:file_selector/file_selector.dart';
```

```dart
import 'package:flutter/material.dart';
import 'package:window_manager/window_manager.dart';

import 'canvas/editor_toolbar.dart';
import 'canvas/paint_canvas.dart';
import 'editor_controller.dart';
import 'editor_menu.dart';
import 'shape/shape_type.dart';
import 'shape_csv.dart';
import 'table_window_controller.dart';

class EditorPage extends StatefulWidget {
  const EditorPage({super.key, required this.title, required this.window});

  final String title;
  final WindowController window;

  @override
  State<EditorPage> createState() => _EditorPageState();
}

class _EditorPageState extends State<EditorPage> with WindowListener {
  late final EditorController _editor;
  late final TableWindowController _table;
  bool _closing = false;

  @override
  void initState() {
    super.initState();
    _editor = EditorController()..addListener(_handleEditorChanged);
    _table = TableWindowController(widget.window, _editor.shapes);
    windowManager.addListener(this);
    unawaited(windowManager.setPreventClose(true));
    _updateTitle(_editor.selectedTool);
  }

  @override
  void dispose() {
    windowManager.removeListener(this);
    _table.dispose();
    _editor
      ..removeListener(_handleEditorChanged)
      ..destroy();
    super.dispose();
  }

  void _handleEditorChanged() {
    setState(() {});
  }

  void _selectTool(ShapeType? tool) {
    _editor.selectTool(tool);
    _updateTitle(tool);
  }

  void _updateTitle(ShapeType? tool) {
    windowManager.setTitle(tool == null ? 'Paint' : 'Paint (${tool.label})');
  }

  Future<void> _openTable() async {
    if (_closing) return;
    await _table.open();
  }

  @override
  void onWindowClose() async {
    if (_closing) return;
    _closing = true;
    try {
      await _table.close();
      await windowManager.destroy();
    } finally {
      _closing = false;
    }
  }

  Future<void> _open() async {
```

```
    const type = XTypeGroup(label: 'CSV', extensions: ['csv']);
    final file = await openFile(acceptedTypeGroups: [type]);
    if (file == null) return;
    final shapes = ShapeCsv.decode(await file.readAsString());
    if (!mounted) return;
    _editor.shapes.replaceAll(shapes);
  }

  Future<void> _save() async {
    const type = XTypeGroup(label: 'CSV', extensions: ['csv']);
    final location = await getSaveLocation(acceptedTypeGroups: [type], suggestedName: 'paint.csv');
    if (location == null) return;
    await File(location.path).writeAsString(ShapeCsv.encode(_editor.shapes.shapes));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // appBar: AppBar(title: Text(widget.title)),
      body: Column(
        children: [
          Align(
            alignment: AlignmentDirectional.centerStart,
            child: EditorMenu(
              selectedTool: _editor.selectedTool,
              onToolSelected: _selectTool,
              onOpen: _open,
              onSave: _save,
              onTable: _openTable,
            ),
          ),
          EditorToolbar(selectedTool: _editor.selectedTool, onToolSelected: _selectTool),
          Expanded(child: PaintCanvas(editor: _editor)),
        ],
      ),
    );
  }
}
```

Лістинг 3 – editor_page.dart

```
import 'package:flutter/foundation.dart';

import 'canvas/canvas_shapes.dart';
import 'shape/shape.dart';
import 'shape/shape_type.dart';

class EditorController extends ChangeNotifier {
  EditorController._();

  static EditorController? _instance;

  factory EditorController() => _instance ??= EditorController._();

  CanvasShapes? _shapes = CanvasShapes();
  ShapeType? _selectedTool;

  bool get isDestroyed => _shapes == null;

  CanvasShapes get shapes {
    final storage = _shapes;
    if (storage == null) {
      throw StateError('EditorController is destroyed');
    }
    return storage;
  }

  ShapeType? get selectedTool => _selectedTool;

  void selectTool(ShapeType? tool) {
    if (_selectedTool == tool) {
      return;
    }
    _selectedTool = tool;
    notifyListeners();
  }
```

```
  void addShape(Shape shape) => shapes.add(shape);

  void destroy() => dispose();

  @override
  void dispose() {
    if (isDestroyed) {
      return;
    }
    _shapes!.shapes.clear();
    _shapes = null;
    _selectedTool = null;
    _instance = null;
    super.dispose();
  }
}
```

Лістинг 4 – editor_controller.dart

```
import 'package:flutter/material.dart';

import 'shape/shape_type.dart';

class EditorMenu extends StatelessWidget {
  const EditorMenu({
    super.key,
    required this.selectedTool,
    required this.onToolSelected,
    required this.onOpen,
    required this.onSave,
    required this.onTable,
  });

  final ShapeType? selectedTool;
  final ValueChanged<ShapeType?> onToolSelected;
  final VoidCallback onOpen;
  final VoidCallback onSave;
  final VoidCallback onTable;

  @override
  Widget build(BuildContext context) {
    return MenuBar(
      children: [
        SubmenuButton(
          menuChildren: [
            MenuItemButton(onPressed: onOpen, child: const Text('Відкрити…')),
            MenuItemButton(onPressed: onSave, child: const Text('Зберегти як…')),
          ],
          child: const Text('Файл'),
        ),
        SubmenuButton(
          menuChildren: [
            for (final ShapeType tool in ShapeType.values)
              CheckboxMenuButton(
                value: selectedTool == tool,
                onChanged: (bool? selected) => onToolSelected(selected == true ? tool : null),
                child: Text(tool.label),
              ),
          ],
          child: const Text('Об\'єкти'),
        ),
        SubmenuButton(
          menuChildren: [
            MenuItemButton(
              onPressed: onTable,
              child: const Text('Таблиця'),
            ),
          ],
          child: const Text('Довідка'),
        ),
      ],
    );
  }
}
```

Лістинг 5 – editor_menu.dart

```dart
import 'dart:async';

import 'package:desktop_multi_window/desktop_multi_window.dart';
import 'package:flutter/services.dart';

import 'canvas/canvas_shapes.dart';
import 'table_bridge.dart';

class TableWindowController {
  TableWindowController(this._mainWindow, CanvasShapes shapes) {
    _bridge = TableBridge(shapes, onChanged: _publishSnapshot);
    _handlerReady = _mainWindow.setWindowMethodHandler(_handleCall);
  }

  final WindowController _mainWindow;
  late final TableBridge _bridge;
  late final Future<void> _handlerReady;
  WindowController? _window;
  Completer<void>? _ready;
  Future<void>? _opening;
  bool _disposed = false;

  Future<dynamic> _handleCall(MethodCall call) async {
    switch (call.method) {
      case 'ready':
        final ready = _ready;
        if (ready != null && !ready.isCompleted) ready.complete();
        return _bridge.snapshot;
      case 'select':
        _bridge.select(call.arguments as int);
        return _bridge.snapshot;
      case 'delete':
        _bridge.delete(call.arguments as int);
        return _bridge.snapshot;
      default:
        throw MissingPluginException('Unknown method: ${call.method}');
    }
  }

  void _publishSnapshot() {
    final window = _window;
    if (window == null || !(_ready?.isCompleted ?? false)) return;
    unawaited(_sendSnapshot(window));
  }

  Future<void> _sendSnapshot(WindowController window) async {
    try {
      await window.invokeMethod('snapshot', _bridge.snapshot);
    } on WindowChannelException {
      //
    }
  }

  Future<void> open() async {
    if (_disposed || _opening != null) return;
    final window = _window;
    if (window != null) {
      await window.show();
      return;
    }
    final operation = _openWindow();
    _opening = operation;
    try {
      await operation;
    } finally {
      _opening = null;
    }
  }

  Future<void> _openWindow() async {
    await _handlerReady;
    final ready = Completer<void>();
    _ready = ready;
    final window = await WindowController.create(WindowConfiguration(
      arguments: _mainWindow.windowId,
      hiddenAtLaunch: true,
    ));
```

```
    _window = window;

    await ready.future.timeout(const Duration(seconds: 15));
    await _sendSnapshot(window);
  }

  Future<void> close() async {
    final opening = _opening;
    if (opening != null) {
      try {
        await opening;
      } catch (_) {
        //
      }
    }
    final window = _window;
    if (window == null) return;
    final closed = onWindowsChanged
        .asyncMap((_) => WindowController.getAll())
        .timeout(const Duration(seconds: 5))
        .firstWhere((windows) => !windows.any((item) => item.windowId == window.windowId));

    closed.ignore();
    try {
      await window.invokeMethod('close').timeout(const Duration(seconds: 5));
    } on WindowChannelException {
      final windows = await WindowController.getAll();
      if (windows.any((item) => item.windowId == window.windowId)) rethrow;
      return;
    }
    await closed;
  }

  void dispose() {
    _disposed = true;
    _mainWindow.setWindowMethodHandler(null);
    _bridge.dispose();
  }
}
```

Лістинг 6 – table_window_controller.dart

```
import 'dart:collection';

import 'canvas/canvas_shapes.dart';
import 'shape/shape.dart';
import 'shape_csv.dart';

class TableBridge {
  TableBridge(this._shapes, {required this.onChanged}) {
    _shapes.addListener(_onShapesChanged);
  }

  final CanvasShapes _shapes;
  final void Function() onChanged;
  final Map<Shape, int> _ids = HashMap.identity();
  int _nextId = 0;
  int _revision = 0;

  int _idFor(Shape shape) => _ids.putIfAbsent(shape, () => _nextId++);

  Map<String, Object?> get snapshot => {
    'revision': _revision,
    'rows': [
      for (final shape in _shapes.shapes)
        {'id': _idFor(shape), 'cells': [
          ShapeCsv.typeOf(shape).label,
          '${shape.start.dx}', '${shape.start.dy}',
          '${shape.end.dx}', '${shape.end.dy}',
        ]},
    ],
    'selectedId': _shapes.selected == null ? null : _idFor(_shapes.selected!),
  };

  Shape? _findShape(int id) {
    for (final shape in _shapes.shapes) {
```

```
        if (_idFor(shape) == id) return shape;
      }
      return null;
    }

    void select(int id) {
      final shape = _findShape(id);
      if (shape != null) _shapes.select(shape);
    }

    void delete(int id) {
      final shape = _findShape(id);
      if (shape == null) return;
      _shapes.select(shape);
      _shapes.deleteSelected();
    }

    void _onShapesChanged() {
      final active = HashSet<Shape>.identity()..addAll(_shapes.shapes);
      _ids.removeWhere((shape, _) => !active.contains(shape));
      _revision++;
      onChanged();
    }

    void dispose() => _shapes.removeListener(_onShapesChanged);
  }
```

Лістинг 7 – table_bridge.dart

```
import 'dart:ui';

import 'package:csv/csv.dart';
import 'package:flutter/foundation.dart';

import 'shape/circle_line_shape.dart';
import 'shape/cube_shape.dart';
import 'shape/dot_shape.dart';
import 'shape/ellipse_shape.dart';
import 'shape/line_shape.dart';
import 'shape/rectangle_shape.dart';
import 'shape/shape.dart';
import 'shape/shape_type.dart';

class ShapeCsv {
  static final _csv = Csv(lineDelimiter: '\n', autoDetect: false);
  static const _header = ['type', 'x1', 'y1', 'x2', 'y2'];

  static ShapeType typeOf(Shape shape) => switch (shape) {
    DotShape() => ShapeType.dot,
    LineShape() => ShapeType.line,
    RectangleShape() => ShapeType.rectangle,
    EllipseShape() => ShapeType.ellipse,
    CircleLineShape() => ShapeType.circleLine,
    CubeShape() => ShapeType.cube,
    _ => throw ArgumentError('Unknown shape: ${shape.runtimeType}'),
  };

  static String encode(Iterable<Shape> shapes) {
    final rows = <List<dynamic>>[_header];
    for (final shape in shapes) {
      final type = typeOf(shape);
      rows.add([type.name, shape.start.dx, shape.start.dy, shape.end.dx, shape.end.dy]);
    }
    return _csv.encode(rows);
  }

  static List<Shape> decode(String text) {
    final rows = _csv.decode(text);
    final result = <Shape>[];
    for (var i = 0; i < rows.length; i++) {
      final fields = rows[i].map((value) => value.toString().trim()).toList();
      if (i == 0 && listEquals(fields, _header)) continue;
      if (fields.length != _header.length) {
        throw FormatException('Запис ${i + 1}: потрібно 5 полів');
      }
      final types = ShapeType.values.where((type) => type.name == fields[0]);
```

```
      final coordinates = fields.skip(1).map(double.tryParse).toList();
      if (types.isEmpty || coordinates.any((value) => value == null || !value.isFinite)) {
        throw FormatException('Запис ${i + 1}: невідомий тип або координати');
      }
      final start = Offset(coordinates[0]!, coordinates[1]!);
      final end = Offset(coordinates[2]!, coordinates[3]!);
      if (types.first == ShapeType.dot && start != end) {
        throw FormatException('Запис ${i + 1}: координати точки не збігаються');
      }
      result.add(Shape.of(types.first, start: start, end: end));
    }
    return result;
  }
}
```

Лістинг 8 – shape_csv.dart

```
import 'package:flutter/material.dart';

import '../shape/shape_type.dart';

class EditorToolbar extends StatelessWidget {
  const EditorToolbar({
    super.key,
    required this.selectedTool,
    required this.onToolSelected,
  });

  final ShapeType? selectedTool;
  final ValueChanged<ShapeType?> onToolSelected;

  Widget _buildButton(ColorScheme colorScheme, ShapeType tool) {
    final selected = selectedTool == tool;
    return IconButton(
      icon: Icon(tool.icon),
      tooltip: tool.label,
      isSelected: selected,
      onPressed: () => onToolSelected(selected ? null : tool),
      style: ButtonStyle(
        backgroundColor: WidgetStatePropertyAll<Color?>(
          selected ? colorScheme.secondaryContainer : null,
        ),
        foregroundColor: WidgetStatePropertyAll<Color?>(
          selected ? colorScheme.onSecondaryContainer : null,
        ),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;
    return Material(
      color: colorScheme.surfaceContainer,
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 4, vertical: 2),
        child: Row(
          children: [
            for (final ShapeType tool in ShapeType.values) _buildButton(colorScheme, tool),
          ],
        ),
      ),
    );
  }
}
```

Лістинг 9 – editor_toolbar.dart

```
import 'package:flutter/material.dart';

import '../editor_controller.dart';
import '../shape/shape.dart';
import '../shape/shape_type.dart';
import 'canvas_shapes.dart';

class PaintCanvas extends StatefulWidget {
```

```
  const PaintCanvas({super.key, required this.editor});

  final EditorController editor;

  @override
  State<PaintCanvas> createState() => _PaintCanvasState();
}

class _PaintCanvasState extends State<PaintCanvas> {
  Offset? _dragStart;
  Offset? _dragCurrent;

  Shape _buildShape(Offset start, Offset end) =>
      Shape.of(widget.editor.selectedTool!, start: start, end: end);

  bool get _isDragTool {
    final tool = widget.editor.selectedTool;
    return tool != null && tool.usesDrag;
  }

  void _handleTapDown(TapDownDetails details) {
    final tool = widget.editor.selectedTool;
    if (tool == null || tool.usesDrag) {
      return;
    }
    final position = details.localPosition;
    widget.editor.addShape(_buildShape(position, position));
  }

  void _handlePanStart(DragStartDetails details) {
    if (!_isDragTool) {
      return;
    }
    setState(() {
      _dragStart = details.localPosition;
      _dragCurrent = details.localPosition;
    });
  }

  void _handlePanUpdate(DragUpdateDetails details) {
    if (!_isDragTool) {
      return;
    }
    setState(() {
      _dragCurrent = details.localPosition;
    });
  }

  void _handlePanEnd(DragEndDetails details) {
    if (!_isDragTool) {
      return;
    }
    final start = _dragStart;
    final current = _dragCurrent;
    setState(() {
      _dragStart = null;
      _dragCurrent = null;
    });
    if (start == null || current == null) {
      return;
    }
    widget.editor.addShape(_buildShape(start, current));
  }

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      behavior: HitTestBehavior.opaque,
      onTapDown: _handleTapDown,
      onPanStart: _handlePanStart,
      onPanUpdate: _handlePanUpdate,
      onPanEnd: _handlePanEnd,
      child: ColoredBox(
        color: Colors.white,
        child: CustomPaint(
          painter: _CanvasPainter(
            storage: widget.editor.shapes,
```

```
            tool: widget.editor.selectedTool,
            dragStart: _dragStart,
            dragCurrent: _dragCurrent,
          ),
          child: const SizedBox.expand(),
        ),
      ),
    );
  }
}

class _CanvasPainter extends CustomPainter {
  const _CanvasPainter({
    required CanvasShapes storage,
    required this.tool,
    required this.dragStart,
    required this.dragCurrent,
  }) : _storage = storage,
       super(repaint: storage);

  final CanvasShapes _storage;
  final ShapeType? tool;
  final Offset? dragStart;
  final Offset? dragCurrent;

  @override
  void paint(Canvas canvas, Size size) {
    canvas.clipRect(Offset.zero & size);
    for (final shape in _storage.shapes) {
      shape.paint(canvas);
    }
    final selected = _storage.selected;
    if (selected != null) {
      selected.drawDashed(
        canvas,
        Paint()
          ..color = Colors.blue
          ..style = PaintingStyle.stroke
          ..strokeWidth = 1.5,
        Path()..addRect(selected.bounds.inflate(6)),
      );
    }
    final start = dragStart;
    final current = dragCurrent;
    if (start == null || current == null || tool == null) {
      return;
    }
    Shape.of(tool!, start: start, end: current).paintPreview(canvas);
  }

  @override
  bool shouldRepaint(covariant _CanvasPainter oldDelegate) =>
      oldDelegate.dragStart != dragStart || oldDelegate.dragCurrent != dragCurrent;
}
```

Лістинг 10 – paint_canvas.dart

```
import 'package:flutter/foundation.dart';

import '../shape/shape.dart';

class CanvasShapes extends ChangeNotifier {
  final List<Shape> shapes = <Shape>[];

  Shape? _selected;
  Shape? get selected => _selected;

  int get count => shapes.length;

  void add(Shape shape) {
    shapes.add(shape);
    notifyListeners();
  }

  void select(Shape? shape) {
    if (shape != null && !shapes.contains(shape)) {
```

```
      throw ArgumentError('Shape is not in the canvas');
    }
    if (_selected == shape) return;
    _selected = shape;
    notifyListeners();
  }

  void deleteSelected() {
    final shape = _selected;
    if (shape == null) return;
    shapes.remove(shape);
    _selected = null;
    notifyListeners();
  }

  void replaceAll(Iterable<Shape> replacement) {
    final newShapes = List<Shape>.of(replacement);
    shapes
      ..clear()
      ..addAll(newShapes);
    _selected = null;
    notifyListeners();
  }
}
```

Лістинг 11 – canvas_shapes.dart

```
import 'package:flutter/material.dart';

enum ShapeType {
  dot('Крапка', icon: Icons.trip_origin, usesDrag: false),
  line('Лінія', icon: Icons.horizontal_rule),
  rectangle('Прямокутник', icon: Icons.crop_square),
  ellipse('Еліпс', icon: Icons.panorama_fish_eye),
  circleLine('Лінія з кружечками', icon: Icons.timeline),
  cube('Каркас куба', icon: Icons.view_in_ar);

  const ShapeType(this.label, {required this.icon, this.usesDrag = true});

  final String label;

  final IconData icon;

  final bool usesDrag;
}
```

Лістинг 12 – shape_type.dart

```
import 'dart:math' as math;

import 'package:flutter/material.dart';

import 'circle_line_shape.dart';
import 'cube_shape.dart';
import 'dot_shape.dart';
import 'ellipse_shape.dart';
import 'line_shape.dart';
import 'rectangle_shape.dart';
import 'shape_type.dart';

abstract class Shape {
  const Shape({required this.start, required this.end});

  static const double _dash = 6;
  static const double _gap = 4;

  final Offset start;
  final Offset end;

  Rect get bounds => Rect.fromPoints(start, end);

  factory Shape.of(ShapeType type, {required Offset start, Offset? end}) =>
      switch (type) {
        ShapeType.dot => DotShape(start: start),
        ShapeType.line => LineShape(start: start, end: end ?? start),
        ShapeType.rectangle => RectangleShape(start: start, end: end ?? start),
```

```
        ShapeType.ellipse => EllipseShape(start: start, end: end ?? start),
        ShapeType.circleLine => CircleLineShape(start: start, end: end ?? start),
        ShapeType.cube => CubeShape(start: start, end: end ?? start),
      };

  void drawDashed(Canvas canvas, Paint paint, Path path) {
    for (final metric in path.computeMetrics()) {
      double distance = 0;
      while (distance < metric.length) {
        final next = math.min(distance + _dash, metric.length);
        canvas.drawPath(metric.extractPath(distance, next), paint);
        distance = next + _gap;
      }
    }
  }

  void paint(Canvas canvas);

  void paintPreview(Canvas canvas);
}
```

Лістинг 13 – shape.dart

```
import 'package:flutter/material.dart';

import 'shape.dart';

class DotShape extends Shape {
  const DotShape({required super.start}) : super(end: start);

  @override
  void paint(Canvas canvas) {
    canvas.drawCircle(start, 3, Paint()..color = Colors.black);
  }

  @override
  void paintPreview(Canvas canvas) {}
}
```

Лістинг 14 – dot_shape.dart

```
import 'package:flutter/material.dart';

import 'shape.dart';

class LineShape extends Shape {
  const LineShape({required super.start, required super.end});

  @override
  void paint(Canvas canvas) {
    canvas.drawLine(start, end, _stroke(Colors.black));
  }

  @override
  void paintPreview(Canvas canvas) {
    drawDashed(
      canvas,
      _stroke(Colors.black),
      Path()
        ..moveTo(start.dx, start.dy)
        ..lineTo(end.dx, end.dy),
    );
  }

  static Paint _stroke(Color color) =>
      Paint()
        ..color = color
        ..style = PaintingStyle.stroke
        ..strokeWidth = 2;
}
```

Лістинг 15 – line_shape.dart

```
import 'package:flutter/material.dart';
```

```dart
import 'shape.dart';

class RectangleShape extends Shape {
  const RectangleShape({
    required super.start,
    required super.end,
    this.filled = true,
  });

  final bool filled;

  Rect get _rect => Rect.fromCenter(
    center: start,
    width: (end.dx - start.dx).abs() * 2,
    height: (end.dy - start.dy).abs() * 2,
  );

  @override
  Rect get bounds => _rect;

  Paint get _fill => Paint()..color = Colors.pink;

  Paint _stroke(Color color) =>
      Paint()
        ..color = color
        ..style = PaintingStyle.stroke
        ..strokeWidth = 2;

  @override
  void paint(Canvas canvas) {
    if (filled) {
      canvas.drawRect(_rect, _fill);
    }
    canvas.drawRect(_rect, _stroke(Colors.black));
  }

  @override
  void paintPreview(Canvas canvas) {
    drawDashed(canvas, _stroke(Colors.black), Path()..addRect(_rect));
  }
}
```

Лістинг 16 – rectangle_shape.dart

```dart
import 'package:flutter/material.dart';

import 'shape.dart';

class EllipseShape extends Shape {
  const EllipseShape({required super.start, required super.end});

  Rect get _rect => Rect.fromPoints(start, end);

  Paint _stroke(Color color) =>
      Paint()
        ..color = color
        ..style = PaintingStyle.stroke
        ..strokeWidth = 2;

  @override
  void paint(Canvas canvas) {
    canvas.drawOval(_rect, _stroke(Colors.black));
  }

  @override
  void paintPreview(Canvas canvas) {
    drawDashed(canvas, _stroke(Colors.black), Path()..addOval(_rect));
  }
}
```

Лістинг 17 – ellipse_shape.dart

```dart
import 'package:flutter/material.dart';

import 'ellipse_shape.dart';
import 'line_shape.dart';
```

```
import 'shape.dart';

class CircleLineShape extends Shape {
  CircleLineShape({required super.start, required super.end})
    : _line = LineShape(start: start, end: end),
      _startDot = _circle(start),
      _endDot = _circle(end);

  static const double _radius = 5;

  final LineShape _line;
  final EllipseShape _startDot;
  final EllipseShape _endDot;

  @override
  Rect get bounds => super.bounds.inflate(_radius);

  static EllipseShape _circle(Offset center) => EllipseShape(
    start: Offset(center.dx - _radius, center.dy - _radius),
    end: Offset(center.dx + _radius, center.dy + _radius),
  );

  @override
  void paint(Canvas canvas) {
    _line.paint(canvas);
    _startDot.paint(canvas);
    _endDot.paint(canvas);
  }

  @override
  void paintPreview(Canvas canvas) {
    final paint =
        Paint()
          ..color = Colors.black
          ..style = PaintingStyle.stroke
          ..strokeWidth = 2;
    drawDashed(
      canvas,
      paint,
      Path()
        ..moveTo(start.dx, start.dy)
        ..lineTo(end.dx, end.dy)
        ..addOval(Rect.fromCircle(center: start, radius: _radius))
        ..addOval(Rect.fromCircle(center: end, radius: _radius)),
    );
  }
}
```

Лістинг 18 – circle_line_shape.dart

```
import 'package:flutter/material.dart';

import 'line_shape.dart';
import 'rectangle_shape.dart';
import 'shape.dart';

class CubeShape extends Shape {
  CubeShape({required Offset start, required Offset end})
    : this._rects(Rect.fromPoints(start, end));

  CubeShape._rects(Rect front)
    : _front = front,
      _back = front.shift(_depth(front)),
      _frontFace = RectangleShape(
        start: front.center,
        end: front.bottomRight,
        filled: false,
      ),
      _backFace = RectangleShape(
        start: _rectAt(front, _depth(front)).center,
        end: _rectAt(front, _depth(front)).bottomRight,
        filled: false,
      ),
      _edges = [
        for (final corner in _corners(front))
          LineShape(start: corner, end: corner + _depth(front)),
```

```
        ],
        super(start: front.topLeft, end: front.bottomRight);

    static const double _depthFactor = 0.35;

    final Rect _front;
    final Rect _back;

    @override
    Rect get bounds => _front.expandToInclude(_back);
    final RectangleShape _frontFace;
    final RectangleShape _backFace;
    final List<LineShape> _edges;

    static Offset _depth(Rect rect) =>
        Offset(rect.width * _depthFactor, -rect.height * _depthFactor);

    static Rect _rectAt(Rect rect, Offset shift) => Rect.fromLTWH(
      rect.left + shift.dx,
      rect.top + shift.dy,
      rect.width,
      rect.height,
    );

    static List<Offset> _corners(Rect rect) =>
        [rect.topLeft, rect.topRight, rect.bottomLeft, rect.bottomRight];

    @override
    void paint(Canvas canvas) {
      _backFace.paint(canvas);
      for (final edge in _edges) {
        edge.paint(canvas);
      }
      _frontFace.paint(canvas);
    }

    @override
    void paintPreview(Canvas canvas) {
      final paint =
          Paint()
            ..color = Colors.black
            ..style = PaintingStyle.stroke
            ..strokeWidth = 2;
      final path = Path()
        ..addRect(_front)
        ..addRect(_back);
      final frontCorners = _corners(_front);
      final backCorners = _corners(_back);
      for (var i = 0; i < frontCorners.length; i++) {
        path
          ..moveTo(frontCorners[i].dx, frontCorners[i].dy)
          ..lineTo(backCorners[i].dx, backCorners[i].dy);
      }
      drawDashed(canvas, paint, path);
    }
  }
```

Лістинг 19 – cube_shape.dart

```
import 'dart:async';

import 'package:desktop_multi_window/desktop_multi_window.dart';
import 'package:flutter/material.dart';
import 'package:window_manager/window_manager.dart';

import 'object_table.dart';
import 'table_connection.dart';

class TableWindow extends StatefulWidget {
  const TableWindow({super.key, required this.window});

  final WindowController window;

  @override
  State<TableWindow> createState() => _TableWindowState();
}
```

```
class _TableWindowState extends State<TableWindow> with WindowListener {
  late final TableConnection _connection;

  @override
  void initState() {
    super.initState();
    _connection = TableConnection(widget.window, widget.window.arguments);
    windowManager.addListener(this);
    unawaited(_start());
  }

  Future<void> _start() async {
    await windowManager.setPreventClose(true);
    if (!mounted) return;
    await _connection.start();
    if (!mounted) return;
    await windowManager.waitUntilReadyToShow(const WindowOptions(
      size: Size(700, 450),
      title: 'Таблиця графічних об’єктів',
    ));
    if (!mounted) return;
    await windowManager.show();
    await windowManager.focus();
  }

  @override
  void onWindowClose() => windowManager.hide();

  @override
  void dispose() {
    windowManager.removeListener(this);
    _connection.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: ListenableBuilder(
        listenable: _connection,
        builder: (context, _) {
          return ObjectTable<int>(
            rows: _connection.rows,
            selected: _connection.selected,
            onSelect: _connection.select,
            onDelete: () {
              final selected = _connection.selected;
              if (selected != null) _connection.delete(selected);
            },
          );
        },
      ),
    );
  }
}
```

Лістинг 20 – table_window.dart

```
import 'dart:async';

import 'package:desktop_multi_window/desktop_multi_window.dart';
import 'package:flutter/foundation.dart';
import 'package:flutter/services.dart';
import 'package:window_manager/window_manager.dart';

import 'object_table.dart';

class TableConnection extends ChangeNotifier {
  TableConnection(this._window, String mainWindowId)
    : _main = WindowController.fromWindowId(mainWindowId);

  final WindowController _window;
  final WindowController _main;
  List<TableRowData<int>> _rows = [];
  int? _selected;
```

```
  int _revision = -1;
  bool _disposed = false;

  List<TableRowData<int>> get rows => _rows;
  int? get selected => _selected;

  Future<void> start() async {
    await _window.setWindowMethodHandler(_handleCall);
    final snapshot = await _main.invokeMethod<Map>('ready');
    if (snapshot != null) _applySnapshot(snapshot);
  }

  Future<dynamic> _handleCall(MethodCall call) async {
    switch (call.method) {
      case 'snapshot':
        _applySnapshot(call.arguments as Map);
      case 'close':
        Timer.run(() => windowManager.destroy());
      default:
        throw MissingPluginException('Unknown method: ${call.method}');
    }
  }

  void _applySnapshot(Map snapshot) {
    final revision = snapshot['revision'] as int;
    if (_disposed || revision < _revision) return;
    final rows = snapshot['rows'] as List;
    _revision = revision;
    _selected = snapshot['selectedId'] as int?;
    _rows = rows.map((value) {
      final row = value as Map;
      return TableRowData<int>(
        key: row['id'] as int,
        cells: (row['cells'] as List).cast<String>(),
      );
    }).toList();
    notifyListeners();
  }

  Future<void> select(int id) => _send('select', id);

  Future<void> delete(int id) => _send('delete', id);

  Future<void> _send(String action, int id) async {
    final result = await _main.invokeMethod<Map>(action, id);
    if (result != null) _applySnapshot(result);
  }

  @override
  void dispose() {
    _disposed = true;
    _window.setWindowMethodHandler(null);
    super.dispose();
  }
}
```

Лістинг 21 – table_connection.dart

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

class TableRowData<T> {
  const TableRowData({required this.key, required this.cells});

  final T key;
  final List<String> cells;
}

class ObjectTable<T> extends StatelessWidget {
  const ObjectTable({
    super.key,
    required this.rows,
    required this.selected,
    required this.onSelect,
    required this.onDelete,
  });
```

```
  final List<TableRowData<T>> rows;
  final T? selected;
  final ValueChanged<T> onSelect;
  final VoidCallback onDelete;

  @override
  Widget build(BuildContext context) {
    return Focus(
      autofocus: true,
      onKeyEvent: (node, event) {
        if (event is KeyDownEvent && event.logicalKey == LogicalKeyboardKey.delete && selected != null)
{
          onDelete();
          return KeyEventResult.handled;
        }
        return KeyEventResult.ignored;
      },
      child: SingleChildScrollView(
        child: SingleChildScrollView(
          scrollDirection: Axis.horizontal,
          child: DataTable(
            columns: const [
              DataColumn(label: Text('Назва')),
              DataColumn(label: Text('x1')),
              DataColumn(label: Text('y1')),
              DataColumn(label: Text('x2')),
              DataColumn(label: Text('y2')),
            ],
            rows: [
              for (final row in rows)
                DataRow(
                  selected: row.key == selected,
                  onSelectChanged: (_) => onSelect(row.key),
                  cells: [for (final cell in row.cells) DataCell(Text(cell))],
                ),
            ],
          ),
        ),
      ),
    );
  }
}
```

Лістинг 22 – object_table.dart

Діаграма

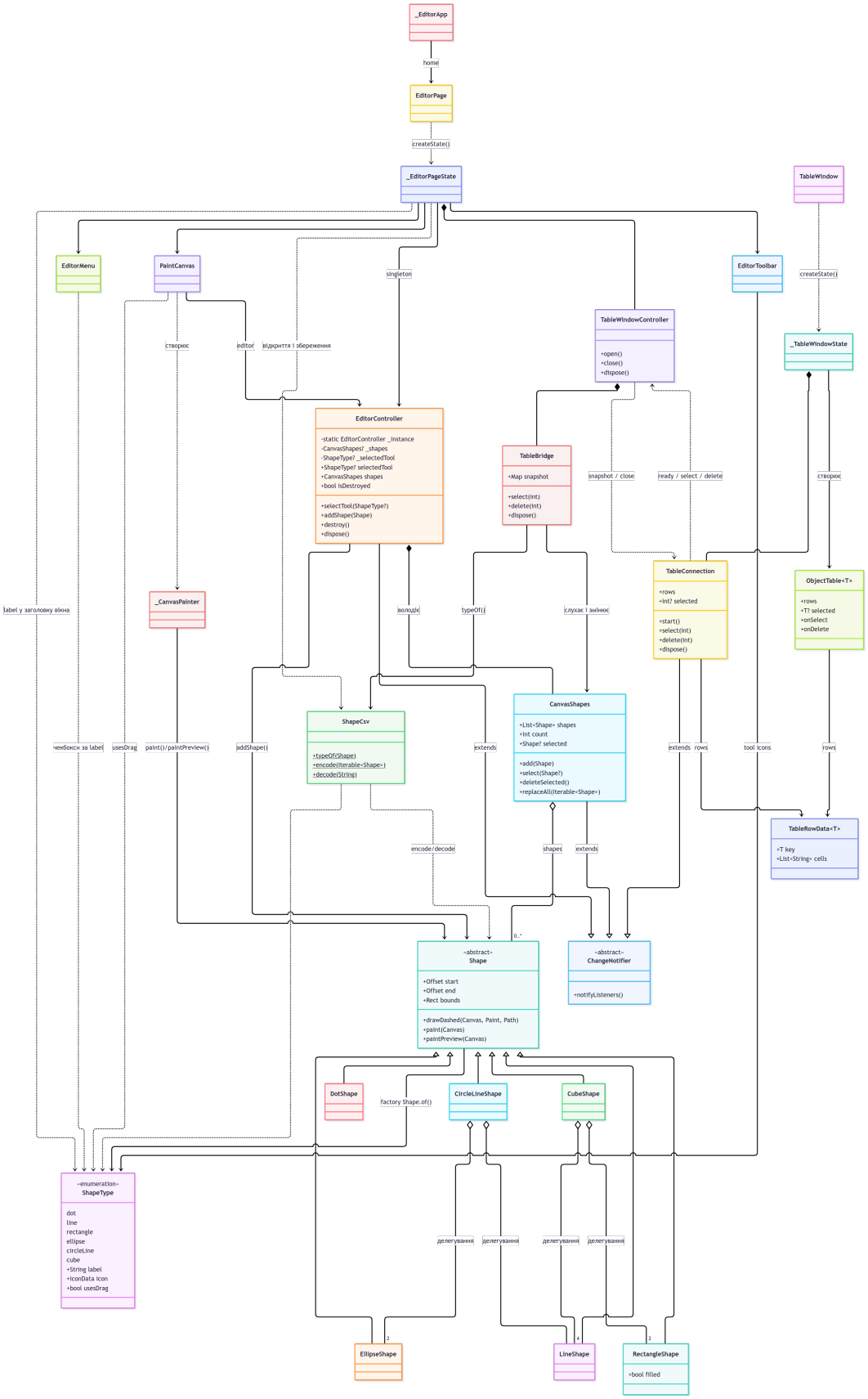


Рисунок 1 – Діаграма класів редактора та вікна таблиці

Приклад файлу об’єктів

Кожен рядок CSV містить тип фігури та координати двох точок. Для прямокутника перша точка задає центр, друга – кут; координати точки збігаються.

```
type,x1,y1,x2,y2
dot,10,130,10,130
line,45,23,400,327
ellipse,123,304,257,412
rectangle,200,200,250,240
```

Скріншоти

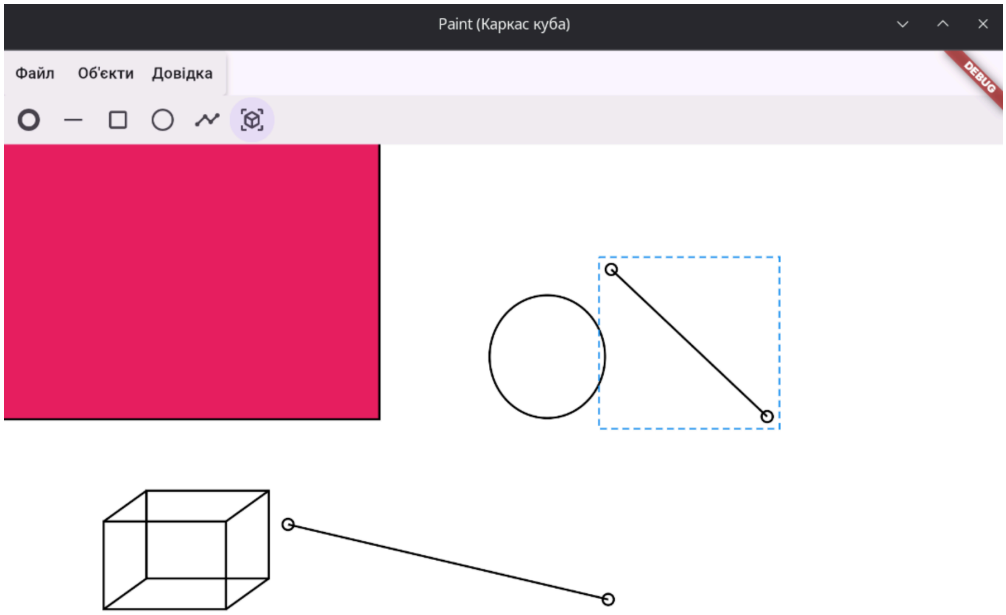


Рисунок 2 – Головне вікно редактора з виділеною фігурою

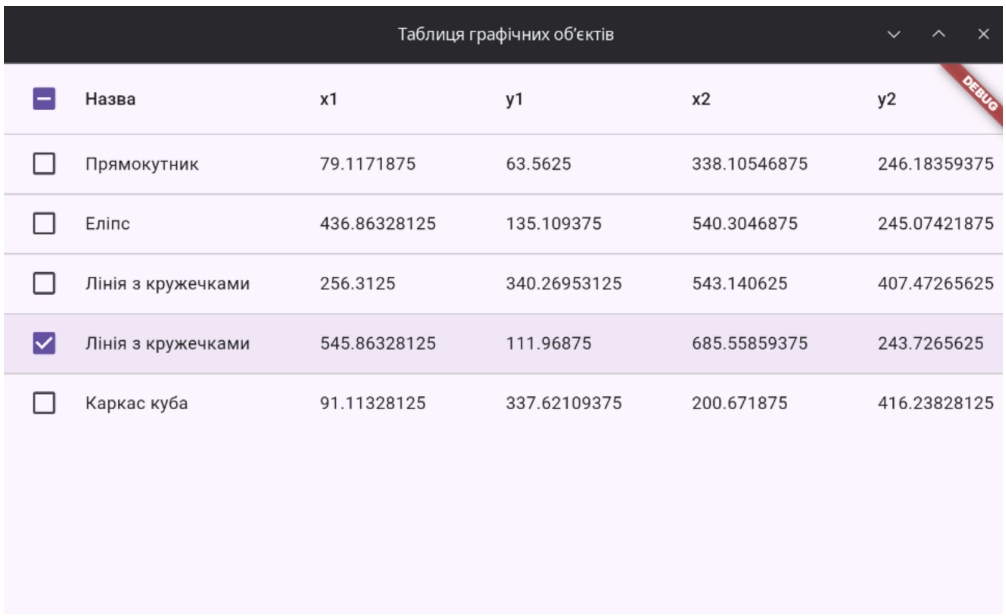


Рисунок 3 – Вікно таблиці з вибраним рядком, що відповідає виділеній фігурі

Висновки

Реалізував багатовіконний інтерфейс редактора, обмін даними між вікнами та збереження й завантаження фігур. Отримав практичний досвід використання Singleton, колбеків і сповіщень про зміни стану.